

Implementation of Cross-Platform Data Transmission Systems: A Comparative Study of Wired Serial and Bluetooth Low Energy (BLE) Bridging using nRF5340

Bhargav K, Harsha B J
Department of Electrical Engineering
IIT Hyderabad

Email: ee25btech11013@iith.ac.in, ee25btech11026@iith.ac.in

Abstract—This paper presents the design and implementation of a cross-platform bidirectional data transfer system, enabling seamless telemetry and file exchange between a Linux host and an Android mobile environment. The study contrasts a baseline wired serial connection against a wireless embedded bridge utilizing the Nordic Semiconductor nRF5340 SoC.

In the wireless implementation, the nRF5340 functions as a protocol converter, bridging USB-CDC (ACM) serial data to Bluetooth Low Energy (BLE) packets via the Nordic UART Service (NUS). To mitigate data corruption caused by control character interpretation in the UART stack, a Base64 encoding scheme was integrated, successfully enabling the transmission of binary assets (images, PDFs) with 100% data integrity.

I. HARDWARE REQUIREMENTS

- **Development Board:** Nordic Semiconductor nRF5340 DK.
- **Smartphone:** Android device with Termux and Serial Bluetooth Terminal App.
- **PC:** Linux machine (Ubuntu).
- **Cable:** Micro USB cable(data transfer and power).

II. SYSTEM MODEL AND HARDWARE ARCHITECTURE

To facilitate cross-platform communication, the system architecture acts as a bridge between standard wired protocols and wireless Bluetooth Low Energy (BLE).

A. System Model

The system model comprises three primary nodes: the Linux Host PC, the nRF5340 Development Kit (acting as the bridge), and the Android Client Device. Data originates from the Linux terminal, passes through a virtual COM port via USB-CDC, is processed by the nRF5340's application core, and is finally transmitted wirelessly using the Nordic UART Service (NUS) over BLE to the smartphone (and vice versa).

B. Hardware Connections

The physical hardware setup is deliberately minimalist, leveraging the onboard Interface MCU (IMCU) of the nRF5340 DK. The Linux PC is connected directly to the **IMCU USB** port of the development kit. This single cable

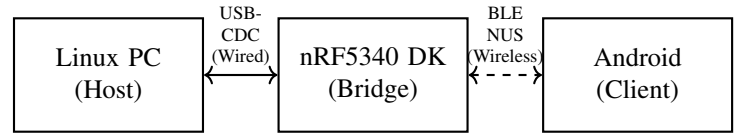


Fig. 1. System Model for Cross-Platform Data Transmission.

serves a dual purpose: it supplies power to the SoC and provides the physical transport layer for the serial UART-to-USB bridge.

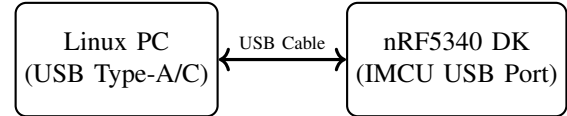


Fig. 2. Hardware Connection Diagram.

III. TRANSFERRING BINARY FILES VIA BASE64

The Nordic UART Service (NUS) is designed for character-based data streams. Attempting to send raw binary files (e.g., .jpg, .png, .pdf) directly will result in data corruption due to control characters and null bytes. To transfer these files, we must encode them into a text-safe format using Base64.

A. 1. Concept Overview

The process involves three stages:

- 1) **Sender (Linux):** Encodes the binary file into a Base64 text string.
- 2) **Transmission:** The text string is sent over the USB-to-Bluetooth bridge.
- 3) **Receiver (Smartphone/PC):** Receives the text string and decodes it back into the original binary format using standard Base64 decoding tools.

B. 2. Technical Constraints

- **Overhead:** Base64 encoding increases the file size by approximately 33%.
- **Speed:** Bluetooth Low Energy (BLE) has a limited bandwidth. The observed speed is around 4.5 kB/second. Transferring large files (e.g., > 1MB) may take several minutes.
- **Buffer:** Ensure the receiving device does not sleep during transfer, as BLE connections may drop if the radio is idle for too long.

IV. NORDIC PROVIDED EXAMPLES FOR UART TEXT TRANSFER

Nordic Semiconductor provides an official sample application for Bluetooth-based serial communication as part of the nRF Connect SDK. The `peripheral_uart` sample, located at:

```
nrf/samples/bluetooth/peripheral_uart
```

demonstrates the use of the **Nordic UART Service (NUS)**, a custom BLE GATT service that emulates a serial port over Bluetooth Low Energy. This sample is designed specifically for text-based data streaming and serves as the official reference implementation for UART-to-BLE bridging on Nordic hardware.

A. What the Official Sample Provides

The stock `peripheral_uart` sample includes the following baseline capabilities:

- Bidirectional text data transfer between a UART interface and a BLE central device.
- Asynchronous UART handling via Zephyr's `uart_async_adapter`.
- A basic two-LED status system: **LED1** blinks to indicate the firmware is running, and **LED2** lights up when a BLE connection is established.
- Optional BLE security via passkey confirmation using the onboard buttons.
- A generic startup message: “*Starting Nordic UART service sample*” printed over UART on boot.

B. Limitations of the Official Sample

While functional, the official sample lacks the following:

- No runtime feedback on how much data has been transferred.
- No visual indication that data is actively being received over BLE (LED2 only reflects connection state, not data activity).
- The startup banner provides no useful runtime configuration information to the developer.

These limitations motivated the enhancements described in the following section.

V. MODIFICATIONS AND IMPROVEMENTS TO THE PERIPHERAL UART SAMPLE

A. Enhancement 1 – Informative Startup Banner

The original startup message printed over UART was a generic placeholder string: “*Starting Nordic UART service sample*”. This was replaced with a concise yet more informative banner that displays the configured buffer size at boot time:

```
NUS Bridge Ready [40]
```

This gives immediate confirmation that the firmware is running and displays the active buffer configuration, which is useful during testing and debugging without exceeding the 40-byte UART transmit buffer limit.

B. Enhancement 2 – BLE Data Activity Indicator via LED

A visual data activity indicator was added using **LED3** (DK_LED3). When the device receives data over the BLE NUS (Nordic UART Service) connection, LED3 briefly illuminates for 100 milliseconds and then turns off automatically.

This was implemented using Zephyr's **delayable work queue** mechanism (`k_work_delayable`), which schedules the LED off event without blocking the BLE callback thread, making it non-intrusive to data throughput.

Why this is better than a simple `k_sleep()`: Calling `k_sleep()` inside a BLE callback can stall the thread and cause missed packets. The work queue approach offloads the LED shutdown to a background task, keeping the data path completely unaffected.

VI. INDUSTRY APPLICATIONS OF NORDIC SOCs

Nordic Semiconductor SoCs are widely adopted in commercial IoT sectors. Notable examples include:

- **Wearables:** The *Ultrahuman Ring Air* and *Dayton Smart-watch* utilize the nRF52 series for biometric tracking (ECG, Sleep Index).
- **Medical Healthcare:** Devices such as *PulseTape* utilize Nordic chips for continuous ECG monitoring, prioritizing low power consumption for extended battery life.
- **Smart Home:** Significantly, the *eCozy 2.0 Smart Thermostat* employs the **nRF5340**—the exact SoC utilized in this project—to handle complex connectivity protocols like Matter and Bluetooth Mesh.

VII. CONCLUSION

This project successfully demonstrated the implementation of a robust, bidirectional wireless data transfer system utilizing the Nordic Semiconductor nRF5340 System-on-Chip. By configuring the nRF5340 as a bridge between a USB-CDC (Serial) interface and the Bluetooth Low Energy (BLE) Nordic UART Service (NUS), we established a seamless communication link between a Linux-based PC and an Android mobile environment.

While the UART protocol is inherently text-based, implementing a Base64 encoding strategy enabled the error-free transmission of complex binary files (images, PDFs) over the

link. The comparative analysis between direct wired connections and the nRF5340-based solution highlights the flexibility of BLE for embedded applications. While wired connections offer higher raw throughput, the nRF5340 implementation provides a reliable, low-power wireless alternative suitable for IoT remote debugging, field telemetry, and file exchange.

APPENDIX A SOFTWARE AND TOOLCHAINS INSTALLATIONS

A. 1. Update & Install System Dependencies

First, update the package manager and install the core dependencies required by the Zephyr build system (CMake, Ninja, Python, etc.).

```
sudo apt update
sudo apt upgrade -y
sudo apt install --no-install-recommends -y \
    git cmake ninja-build gperf \
    ccache dfu-util device-tree-compiler \
    wget \
    python3-dev python3-pip \
    python3-setuptools python3-tk \
    python3-wheel xz-utils file \
    make gcc gcc-multilib g++-multilib \
    libsdl2-dev
```

B. 2. Set up Python Virtual Environment & Install West

Modern Linux distributions enforce PEP 668, which restricts global ‘pip’ installations. The Zephyr build system requires a dedicated Python virtual environment.

```
# Create a virtual environment in the home directory
python3 -m venv ~/zephyr-venv

# Activate the virtual environment
source ~/zephyr-venv/bin/activate

# Install west inside the virtual environment
pip3 install -U west
```

Note: You must run `source ~/zephyr-venv/bin/activate` every time you open a new terminal to build the project.

C. 3. Initialize nRF Connect SDK (v3.2.2)

Create a workspace directory and initialize the specific SDK version used for this project.

```
mkdir ~/ncs
cd ~/ncs

# Initialize the SDK pointing to version v3.2.2
west init -m
https://github.com/nrfconnect/sdk-nrf
--mr v3.2.2

# Download the code
west update
```

```
# Export CMake package for Zephyr
west zephyr-export
```

D. 4. Install Python Dependencies

With the virtual environment activated, install the specific Python requirements for the Zephyr kernel, the nRF SDK, and the bootloader.

```
pip3 install -r
zephyr/scripts/requirements.txt
pip3 install -r nrf/scripts/requirements.txt
pip3 install -r
bootloader/mcuboot/scripts/requirements.txt
```

E. 5. Install the Zephyr SDK Toolchain

Download and install the cross-compiler toolchain (Zephyr SDK 0.16.8) required to compile code for the nRF5340 on NCS v3.2.x.

```
# Download the Zephyr SDK Bundle
cd ~
wget
https://github.com/zephyrproject-rtos/sdk-ng/
releases/download/v0.16.8/
zephyr-sdk-0.16.8_linux-x86_64.tar.xz

# Extract the archive
tar xvf
zephyr-sdk-0.16.8_linux-x86_64.tar.xz

# Run the setup script
cd zephyr-sdk-0.16.8
./setup.sh -c
```

F. 6. Install nRF Command Line Tools

Finally, install the J-Link drivers and `nrfjprog` to enable flashing the board over USB.

```
# Navigate to the download directory
cd ~/Downloads

# Install the .deb package (replace filename with actual version)
sudo dpkg -i nrf-command-line-tools*.deb

# Fix any missing dependencies
sudo apt install -f
```

G. Verification

Verify the installation by checking the versions of the flashing and build tools.

```
nrfjprog --version
west --version
```

APPENDIX B PROJECT EXECUTION PROCEDURE

This section details the procedure to compile, flash, and test the `peripheral_uart` sample application using the command line interface.

A. 1. Workspace Initialization

Before building, ensure the workspace is correctly set up. If you have not created a specific project folder yet, navigate to: Target Board: nrf5340dk/nrf5340/cpuapp.

```
cd ~/ncs/v3.2.2
source zephyr/zephyr-env.sh
```

B. 2. Building the Firmware

Important Note: Before building, ensure that the default `main.c` file in the sample directory has been replaced with the custom `main.c` code provided in the project repository¹. Target Board: nrf5340dk/nrf5340/cpuapp.

```
west build -b nrf5340dk/nrf5340/cpuapp
nrf/samples/bluetooth/peripheral_uart
```

C. 3. Flashing the Board

Once the build completes successfully, flash the binary to the nRF5340 development kit.

- 1) Connect the nRF5340 DK to the PC via the **IMCU USB** port (Interface MCU).
- 2) Run the flash command:

```
cd /nrf/samples/bluetooth/peripheral_uart
west flash
```

If successful, the terminal will display Flashing completed.

APPENDIX C

FILE TRANSFER : LINUX TERMINAL & ANDROID SERIAL APP

This section details the procedure to verify bidirectional text transfer using a Linux terminal on the PC side and the **Serial Bluetooth Terminal** app on an Android device.

A. 1. Linux Setup (Receiver/Sender)

On the Ubuntu machine, we interact with the nRF5340 board through its virtual serial port.

- 1) Connect the nRF5340 DK to the PC via the **IMCU USB** port.
- 2) Open a terminal window and list the USB devices to identify the board (usually `tttACM0` or `tttACM1`).

```
ls /dev/ttyACM*
```

- 3) Open the serial connection using `minicom` with a baud rate of **115200**.

```
# Replace ttyACM1 with your actual port
sudo minicom -D /dev/ttyACM1 -b 115200
```

- 4) The terminal screen will clear and wait for input. (Press **Reset** on the board if you want to see the boot log).

¹https://github.com/gadepall/nordic/blob/98d3648fbc7c7dbd8c8d82ef259fdef46a244059/File_Transfer/codes/main.c

B. 2. Android Setup (Serial Bluetooth Terminal)

We use a dedicated terminal app to act as the Bluetooth Central device.

- 1) **Install App:** Download and install **Serial Bluetooth Terminal** (by Kai Morich) from the Google Play Store.
- 2) **Open App:** Launch the application.
- 3) **Scan:**
 - Tap the menu (three lines) in the top-left corner.
 - Select **Devices**.
 - Tap the **Bluetooth LE** tab (important: do not use the Classic Bluetooth tab).
 - Tap **Scan** at the top.
- 4) **Connect:** Locate the device named **Nordic_UART_Service** in the list and tap it to connect. The terminal screen should show "Connected".

C. 3. Verifying File Transfer (Text Streaming)

This section details the procedure to transfer entire text files between the Android device and the Linux machine using the Nordic UART Service.

1) *Smartphone to Linux (Sending a File):* To send a text file stored on the phone and save it directly to the Linux host:

- 1) Ensure the devices are connected via the **Serial Bluetooth Terminal** app.
- 2) On the Linux terminal, instruct `minicom` to capture the incoming serial data to a file by pressing **Ctrl+A**, followed by **L**.
- 3) When prompted, enter a filename (e.g., `received_file.txt`) and press **Enter**.
- 4) On the Android app, tap the **Menu** icon (three lines) or the **Upload/Folder** icon.
- 5) Select **Send File**, then browse and select the desired `.txt` file.
- 6) **Result:** The content of the file will stream into the open `minicom` session and be written directly to the capture file on the PC.
- 7) Once the transfer is complete, safely close and save the file in `minicom` by pressing **Ctrl+A**, followed by **L**, and selecting **Close**.

2) *Linux to Smartphone (Sending a File):* To send a text file from the Linux machine to the phone, we cannot just type in the terminal. We must pipe the file directly to the device port.

- 1) Open a **new** terminal window (keep the `minicom` session open in the first window to monitor the connection).
- 2) Create a dummy text file to send:

```
echo "This is a test file sent from
Linux." > test_file.txt
```

- 3) Use the `cat` command to redirect the file into the USB serial port:

```
# Replace ttyACM1 with your actual port
sudo sh -c 'cat test_file.txt >
/dev/ttyACM1'
```

- 4) **Result:** The content of `test_file.txt` will appear in the **Serial Bluetooth Terminal** app on your phone (usually in green).
- 5) **Save:** To save the text as a file to your phone, tap the three-dot menu, select Data, and choose Save. To update the save location, go to Settings > Misc. and modify the Save+log folder path.

APPENDIX D

BINARY FILE TRANSFER: SMARTPHONE TO LINUX HOST

This section details the specific procedure for transmitting raw binary assets (such as `.jpg` or `.png` or `.pdf` files) from the Android mobile environment to the Linux host. To circumvent the Nordic UART Service's (NUS) inability to handle raw binary control characters, a local Base64 encoding workflow was utilized on the smartphone prior to transmission.

A. 1. Base64 Encoding via Termux (Android)

Android does not natively provide a user-accessible command-line interface. Therefore, **Termux**, a robust terminal emulator for Android, was employed to handle the file conversion locally on the device.

- 1) Open Termux and navigate to the directory containing the source binary file (ensuring Termux has storage permissions).
- 2) Execute the `base64` command to encode the binary file into a plain text format.

```
# Encode an image into a Base64 text file
  on the phone
base64 original_image.jpg >
  encoded_payload.txt

# Verify the text file was created
  successfully
ls -lh encoded_payload.txt
```

B. 2. BLE Transmission via Serial Terminal App

Once the binary file is safely converted to text, it can be transmitted without risk of UART data corruption.

- 1) Launch the **Serial Bluetooth Terminal** application on the smartphone.
- 2) Connect to the nRF5340 board via the **Nordic UART Service**.
- 3) Navigate to the app's attachment/menu interface and select **Send File**.
- 4) Select the `encoded_payload.txt` file generated by Termux.
- 5) The app will stream the Base64 text over the BLE link to the nRF5340, which bridges it to the PC via USB-CDC.

C. 3. Stream Capture and Decoding (Linux PC)

On the receiving end, the Linux host must capture the incoming serial stream and decode it back into its original binary form.

- 1) **Capture the Stream:** Open a terminal on the Linux PC and use the `minicom` command to listen to the

nRF5340's serial port, redirecting the output to a new text file which has been mentioned in Section ??.

- 2) **Decode to Binary:** Utilize the native Linux `base64` utility to reverse the encoding process and restore the file.

```
# Decode the captured text back into the
  original image
base64 -d received_payload.txt >
  restored_image.jpg
```

The image at the sender and the receiver's end has been found out to be the same. This shows 100% data integrity even for images and pdf.

APPENDIX E

FILE TRANSFER BETWEEN 2 DEVICES VIA BLUETOOTH WITHOUT USING NRF5340 BLE

Switch on Bluetooth on both devices.

A. Installing the Packages

```
sudo apt update
sudo apt install bluez bluetooth
  bluetooth-obex obexftp gnome-bluetooth
sudo systemctl start bluetooth
```

B. Connecting the sender to the receiver

On the sender device:

```
bluetoothctl scan on
```

Note the MAC address: AA:BB:CC:DD:EE:FF

```
bluetoothctl
```

Inside the interactive terminal:

```
pair AA:BB:CC:DD:EE:FF
trust AA:BB:CC:DD:EE:FF
connect AA:BB:CC:DD:EE:FF
exit
```

```
bluetooth-sendto --device=AA:BB:CC:DD:EE:FF
  /path/to/file1.txt /path/to/file2.jpg
  /path/to/file3.png
```

To automate this process:

Save the following as `bt-multi-send.sh`

```
#!/bin/bash
DEVICE="AA:BB:CC:DD:EE:FF" # Replace with
  MAC Address
DIR_PATH="$1" # Pass dir as arg, e.g.,
  ./bt-multi-send.sh /path/to/files

if [ -z "$DIR_PATH" ]; then
  echo "Usage: $0 <directory>"
  exit 1
fi

# Ensure paired
bluetoothctl connect $DEVICE > /dev/null
2>&1
```

```
for file in "$DIR_PATH"/*; do
  if [ -f "$file" ]; then
    echo "Sending $file..."
    bluetooth-sendto --device=$DEVICE
    "$file"
  fi
done
```

Make it executable:

```
chmod +x bt-multi-send.sh
```

Run this command:

```
./bt-multi-send.sh /path/to/files
```

REFERENCES

- [1] Nordic Semiconductor, *nRF5340 Objective Product Specification v1.3*, 2023. [Online]. Available: <https://infocenter.nordicsemi.com/>
- [2] Zephyr Project, *Zephyr Project Documentation*. [Online]. Available: <https://docs.zephyrproject.org/>
- [3] Bluetooth Special Interest Group (SIG), *Bluetooth Core Specification v5.0*, 2016.